KARNAVATI UNIVERSITY
NAAC GRADE A+

KARNAVATI UNIVERSITY
UNITEDWORLD INSTITUTE OF TECHNOLOGY (UIT)

MINI PROJECT REPORT

ON

"MINI COMPILER FOR IF-ELSE STATEMENTS"

Submitted as a part of the course

Compiler Design

Submitted By

Student Name	Enrollment No.
Om Patel	202307020079
Pallav Patel	202307020080
Rudra Patel	202307020082

Program: B.Tech CSE-AIML
Semester: VII

Under the Guidance of

Ms Neha Minocha
Assistant Professor
Department of Computer Science & Engineering Technology

Karnavati University, Gandhinagar

TABLE OF CONTENTS

1. Abstract 3
2. Introduction 3
3. Overview of Project 4
4. Technology Used 8
5. Flow Chart of the Project 8
6. Screenshots of the Project 9
7. Results and Conclusion 10

LIST OF FIGURES

Figure 3.1: Token definitions and operator matching in lexer.py 5
Figure 3.2: parse_if() in parser.py 5
Figure 3.3: Symbol table scope handling in semantic.py 5
Figure 3.4: TAC generation for if-else with labels in icg.py 6
Figure 3.5: Constant propagation and folding in optimizer.py 6
Figure 3.6: The compile_source() pipeline in pipeline.py 6
Figure 5.1: Flow chart of the mini compiler 7
Figure 6.1: Web UI with a nested if-else program and its output 8
Figure 6.2: Tokens generated for nested if-else program 8
Figure 6.3: Syntax tree showing the dangling else rule 8
Figure 6.4: Symbol table with a shadowed variable 8
Figure 6.5: TAC before and after optimization 8
Figure 6.6: Error case: undeclared variable 8
Figure 7.1: Result of running all 86 unit tests 9

CHAPTER 1: ABSTRACT

We built a mini compiler in Python for a small C-like language centred on if-else statements. Every program passes through six phases we wrote by hand: a lexer, a recursive descent parser, a semantic analyzer with a scoped symbol table, a three-address code generator, an optimizer, and an interpreter that runs the result. A Flask web page compiles code with one click and shows the output, the tokens and the status of each phase, and errors come back with their phase and line number instead of crashing the page. All 86 unit tests pass, and the project shows how an if-else turns into jumps and how much of that code disappears when values are known at compile time.

CHAPTER 2: INTRODUCTION

Every C or Java program we have written went through a compiler, which checks the code, reports mistakes with their line and translates it in phases. An assignment like x = a + b maps almost directly to one instruction, but an if-else must become conditional jumps and labels. It also brings in the dangling else, nested scopes and short-circuit && and ||, so even a small language covers the hard parts.

2.1 Problem Statement

Build a compiler for a C-like language with if, if-else, else-if and nested if that analyses the program, generates, optimizes and runs three-address code, and reports errors with phase and line number.

2.2 Objectives

- Tokenize the source with line and column numbers and report invalid characters.
- Parse a BNF grammar by recursive descent, build an AST and handle the dangling else.
- Check declarations, block scopes and types with a scoped symbol table.
- Generate TAC with labels and short-circuit jumps, then optimize and run it.
- Show results and errors in a web UI that never crashes, and unit test every phase.

CHAPTER 3: OVERVIEW OF PROJECT

The compiler/ package has one module per phase: lexer.py, parser.py (with the tree classes in ast_nodes.py), semantic.py, icg.py (with the instruction format in tac.py), optimizer.py and executor.py, while errors.py holds the shared CompilerError and CompilerWarning classes. compile_source() in pipeline.py runs the phases in order for both the Flask app and cli.py. Each phase only reads the output of the one before it, so we could write and test them one at a time. The web page is templates/index.html with static/css/style.css and static/js/app.js, and samples/ holds 22 valid and 9 error programs that the page loads from its dropdown.

3.1 The Language and Its Tokens

Our language is a small subset of C: int and float, + - * / %, < > <= >= == !=, && || !, if, else-if, nested blocks, // comments and print. This program prints Marks: 72 Grade: 3:

```
int marks = 72;
float average = 81.5;
int grade;                    // starts at 0
if (marks >= 90) grade = 1;
else if (marks >= 75 && average > 80.0) grade = 2;
else grade = 3;
print("Marks:", marks, "Grade:", grade);
```

Table 3.1 lists the token types our lexer produces.

Table 3.1: Token types

Token Type	Lexemes	Token Type	Lexemes
KEYWORD	int float if else print	ARITH_OP	+ - * / %
IDENTIFIER	marks, _x1	REL_OP	< > <= >= == !=
INT_LITERAL	72	LOGICAL_OP	&& || !
FLOAT_LITERAL	81.5	ASSIGN	=
STRING_LITERAL	"Grade:"	LPAREN, RPAREN	( )
SEMICOLON, COMMA	; ,	LBRACE, RBRACE	{ }
EOF	end of input		

3.2 Grammar in BNF

The grammar sits at the top of parser.py. { x } means zero or more and [x] means optional; the expression rules set the precedence from || up to the unary operators.

```
<program>    ::= { <statement> } EOF
<statement>  ::= <declaration> | <assignment> | <if_stmt> | <print_stmt> | <block>
<declaration> ::= <type> IDENTIFIER [ "=" <expression> ] ";"
<type>       ::= "int" | "float"
<assignment> ::= IDENTIFIER "=" <expression> ";"
<if_stmt>    ::= "if" "(" <expression> ")" <statement> [ "else" <statement> ]
<print_stmt> ::= "print" "(" [ <print_arg> { "," <print_arg> } ] ")" ";"
<print_arg>  ::= STRING_LITERAL | <expression>
<block>      ::= "{" { <statement> } "}"
<expression> ::= <logic_or>
<logic_or>   ::= <logic_and> { "||" <logic_and> }
<logic_and>  ::= <equality> { "&&" <equality> }
<equality>   ::= <relational> { ( "==" | "!=" ) <relational> }
<relational> ::= <additive> { ( "<" | ">" | "<=" | ">=" ) <additive> }
<additive>   ::= <term> { ( "+" | "-" ) <term> }
<term>       ::= <unary> { ( "*" | "/" | "%" ) <unary> }
<unary>      ::= ( "!" | "-" | "+" ) <unary> | <primary>
<primary>    ::= INT_LITERAL | FLOAT_LITERAL | IDENTIFIER | "(" <expression> ")"
```

3.3 Lexical Analyzer

lexer.py reads one character at a time, skips spaces and comments, and records each token's line and column. Two-character operators are checked first (Figure 3.1), otherwise <= would be read as < and =. The lexer collects every error, such as @, 12abc or an unclosed string, instead of stopping.

```
13  KEYWORDS = {"int", "float", "if", "else", "print"}

16  TWO_CHAR_OPERATORS = {
17      "<=": "REL_OP",
18      ">=": "REL_OP",
19      "==": "REL_OP",
20      "!=": "REL_OP",
21      "&&": "LOGICAL_OP",
22      "||": "LOGICAL_OP",
23  }

197     def read_operator(self):
198         line, column = self.line, self.column
199         pair = self.peek() + self.peek(1)
200         if pair in TWO_CHAR_OPERATORS:
201             self.advance()
202             self.advance()
203             self.add_token(TWO_CHAR_OPERATORS[pair], pair, line, column)
204             return
```

Figure 3.1: Token definitions and operator matching in lexer.py

3.4 Syntax Analyzer

parser.py has one method per grammar rule. The dangling else was our first real problem: in if (a) if (b) x = 1; else x = 2; the else could belong to either if. parse_if() takes the else right after the then-branch (Figure 3.2), so it binds to the nearest if, as in C.

```
150     def parse_if(self):
151         if_token = self.advance()
152         self.expect("LPAREN", None, "expected '(' after 'if'")
153         condition = self.parse_expression()
154         if not self.check("RPAREN"):
155             hint = " (did you mean '=='?)" if self.check("ASSIGN") else ""
156             raise self.error(f"expected ')' after if condition, found {describe(self.current())}{hint}")
157         self.advance()
158 
159         then_branch = self.parse_branch("if")
160         else_branch = None
161         if self.check("KEYWORD", "else"):  # binds to this (the nearest) if
162             self.advance()
163             else_branch = self.parse_branch("else")
164         return If(condition, then_branch, else_branch, if_token.line, if_token.column)
```

Figure 3.2: parse_if() in parser.py

3.5 Semantic Analyzer

semantic.py keeps a stack of scopes, one per { } block. declare() checks only the innermost scope, so a redeclaration in one block is an error but shadowing is allowed, and lookup() searches outwards (Figure 3.3). Undeclared names and % on floats are errors; unused variables are warnings.

```
77      def declare(self, node):
78          scope_name, table = self.scopes[-1]
79          if node.name in table:
80              first = table[node.name]
81              self.error(f"redeclaration of '{node.name}' in the same scope (first declared at line {first.line})",
82                         node)
83              return None
84          symbol = Symbol(node.name, node.var_type, scope_name, len(self.scopes) - 1, node.line, node.column)
85          table[node.name] = symbol
86          self.symbols.append(symbol)
87          return symbol
88 
89      def lookup(self, name):
90          """Search from the innermost scope outwards."""
91          for _, table in reversed(self.scopes):
92              if name in table:
93                  return table[name]
94          return None
```

Figure 3.3: Symbol table scope handling in semantic.py

3.6 Intermediate Code Generator

icg.py emits three-address code (TAC) with temporaries t1, t2 and labels L1, L2. gen_If() lays out the condition jumps, the then-branch, a goto past the else, and the else-branch (Figure 3.4). Short-circuit jumps took us a few tries: for a && b a middle label skips b when a is false, so if (d != 0 && n / d > 2) never divides by zero.

```
144     def gen_If(self, node):
145         true_label = self.new_label()
146         false_label = self.new_label()
147         self.gen_condition(node.condition, true_label, false_label)
148         self.emit("label", label=true_label)
149         self.gen_statement(node.then_branch)
150         if node.else_branch is None:
151             self.emit("label", label=false_label)
152         else:
153             end_label = self.new_label()
154             self.current_line = node.line
155             self.emit("goto", label=end_label)
156             self.emit("label", label=false_label)
157             self.gen_statement(node.else_branch)
158             self.emit("label", label=end_label)
```

Figure 3.4: TAC generation for if-else with labels in icg.py

3.7 Code Optimizer

optimizer.py repeats its passes until nothing changes. Constants are propagated inside a basic block and folded (Figure 3.5), so if 1 goto L1 becomes goto L1 and the dead branch is removed. Division by zero is left unfolded so it is still reported at run time.

```
        for field in ("arg1", "arg2"):
            operand = getattr(instr, field)
            if isinstance(operand, str) and operand in known:
                setattr(instr, field, known[operand])
                changed = True

        try:
            if instr.kind == "binop" and is_const(instr.arg1) and is_const(instr.arg2):
                value = apply_binary(instr.op, instr.arg1, instr.arg2)
                instr.kind, instr.arg1, instr.op, instr.arg2 = "copy", value, None, None
                changed = True
            elif instr.kind == "unop" and is_const(instr.arg1):
                value = apply_unary(instr.op, instr.arg1)
                instr.kind, instr.arg1, instr.op = "copy", value, None
                changed = True
            elif instr.kind == "if" and instr.op and is_const(instr.arg1) and is_const(instr.arg2):
                instr.arg1 = apply_binary(instr.op, instr.arg1, instr.arg2)
                instr.op, instr.arg2 = None, None
                changed = True
        except ZeroDivisionError:
            pass  # leave it for the executor to report at run time
```

Figure 3.5: Constant propagation and folding in optimizer.py

3.8 Execution and the Compile Pipeline

executor.py runs the optimized TAC with a program counter and a dictionary of variables. compile_source() (Figure 3.6) never raises: a failed phase marks the rest as skipped and even an internal bug comes back as an error message, so the web page never shows a traceback.

```
def compile_source(source):
    """Compile and run `source`. Always returns a dict; never raises."""

    status = {phase: "skipped" for phase in PHASES}
    started = time.perf_counter()

    try:
        _run_phases(source, result, status)
    except Exception as exc:  # a bug in the compiler itself; never show a traceback
        current = next(phase for phase in PHASES if status[phase] == "skipped")
        message = ("program is nested too deeply to compile" if isinstance(exc, RecursionError)
                   else f"internal compiler error: {exc}")
        _fail(result, status, current, [CompilerError(current, message)])

    result["success"] = result["failed_phase"] is None
    result["phases"] = [{"name": phase, "status": status[phase]} for phase in PHASES]
    result["time_ms"] = round((time.perf_counter() - started) * 1000, 2)
    return result
```

Figure 3.6: The compile_source() pipeline in pipeline.py

CHAPTER 4: TECHNOLOGY USED

Table 4.1: Technologies used

Technology	Purpose
Python 3.11	Language for all six compiler phases, the CLI and the tests
Flask 3.1	Web server for the page and the /api/samples and /api/compile routes
HTML5 and CSS3	Editor with line numbers, tabs, phase bar, light and dark theme
JavaScript	Sends the code with fetch() and shows output, tokens and errors
unittest	86 automated tests for all phases, samples and routes
argparse	Options of cli.py that print tokens, tree, symbol table and TAC
VS Code	Writing, running and debugging the code

We chose Python because each phase fits in a few hundred readable lines, so we could focus on compiler ideas. The compiler uses only the standard library; Flask is the one entry in requirements.txt.

CHAPTER 5: FLOW CHART OF THE PROJECT

The JavaScript posts the code to /api/compile, which calls compile_source(). After each phase we check for errors: with none the next phase starts, with one the rest are skipped and the error returns with its phase and line number. The page then shows the output, tokens and phase colours.

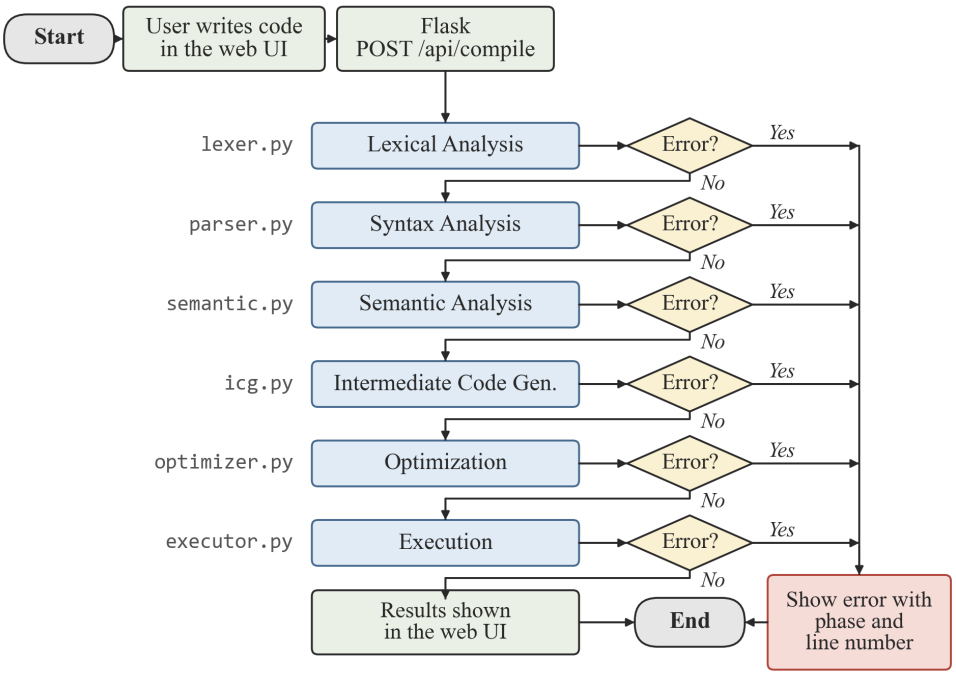


Figure 5.1: Flow chart of the mini compiler

CHAPTER 6: SCREENSHOTS OF THE PROJECT

Figure 6.1 shows the page after loading the Nested If Else sample and clicking Compile: the output is 2024 is a leap year and all six phases are green.

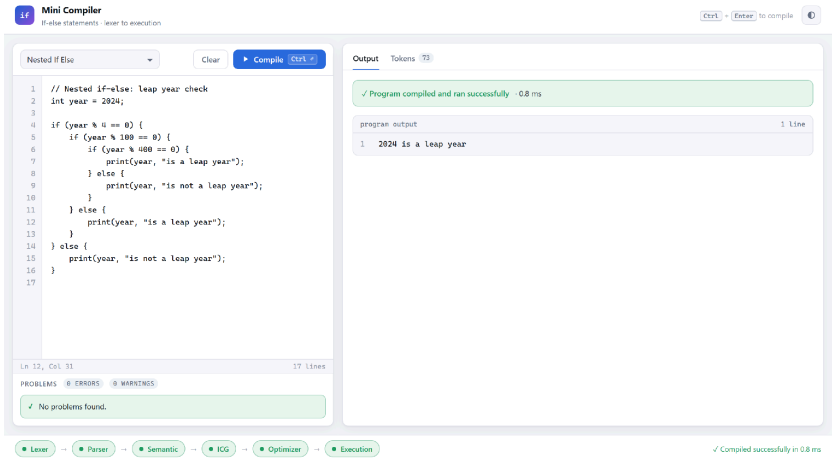


Figure 6.1: Web UI with a nested if-else program and its output

Figure 6.2 is the Tokens tab. The page has only Output and Tokens, so Figures 6.3 to 6.5 come from cli.py on our dangling-else sample, where the else sits under the inner if (y > 0).

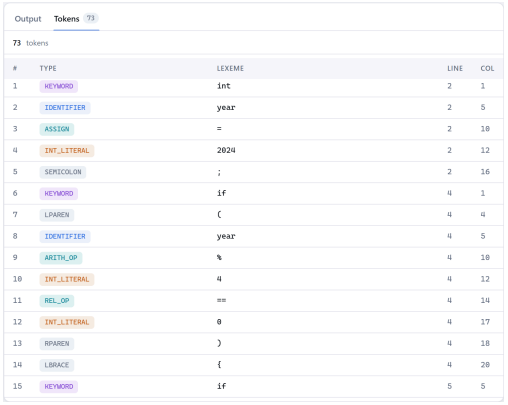


Figure 6.2: Tokens generated for nested if-else program

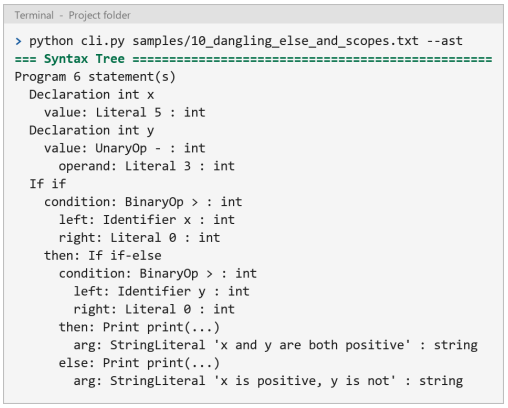


Figure 6.3: Syntax tree showing the dangling else rule

Figure 6.4 has two value entries because the inner block shadows the outer one, and in Figure 6.5 the 20 TAC lines shrink to 9. Figure 6.6 shows the undeclared variable errors with Semantic in red.

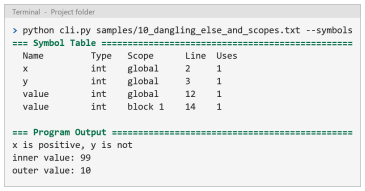


Figure 6.4: Symbol table with a shadowed variable

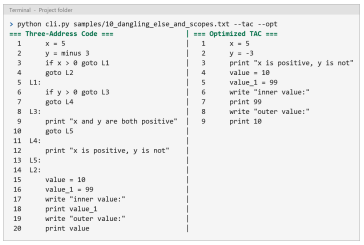


Figure 6.5: TAC before and after optimization

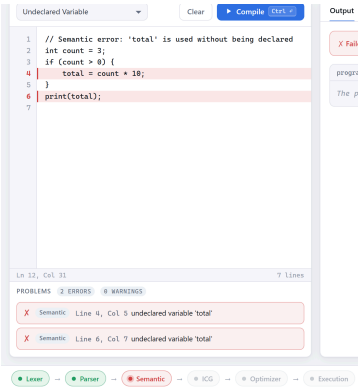


Figure 6.6: Error case: undeclared variable

CHAPTER 7: RESULTS AND CONCLUSION

7.1 Results

Our 86 unit tests cover each phase, the pipeline, all 31 sample programs and the Flask routes.

Table 7.1 compares expected and actual results for some samples.

Table 7.1: Test cases from the sample programs

Input (sample file)	Expected	Actual	Pass/Fail
02_if_else	27 is odd	27 is odd	Pass
04_nested_if_else	2024 is a leap year	2024 is a leap year	Pass
22_safe_division (d = 0)	Skipped: d is 0, so n / d was never computed	Skipped: d is 0, so n / d was never computed	Pass
errors/01_lexical_error	Lexer: invalid character	Lexer, line 3: invalid character '@'	Pass
errors/02_missing_semicolon	Parser: expected ';'	Parser, line 4: expected ';' after expression	Pass
errors/04_undeclared_variable	Semantic: undeclared variable	Semantic, line 4: undeclared variable 'total'	Pass
errors/08_division_by_zero	Execution: division by zero	Execution, line 5: division by zero	Pass

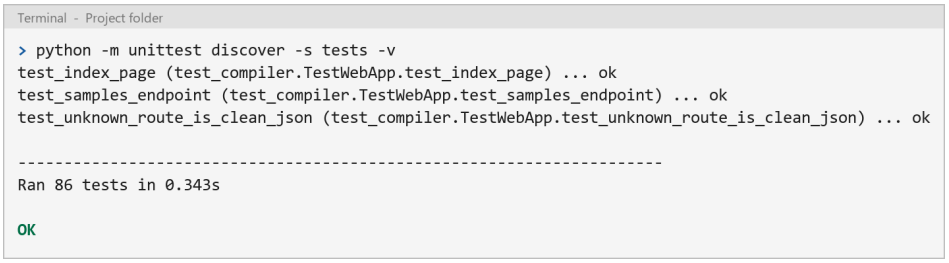


Figure 7.1: Result of running all 86 unit tests

7.2 Conclusion and Future Scope

We built a working compiler for if-else programs that covers every phase from tokens to running code and explains errors instead of crashing. Its limits: only int and float, no loops or functions, TAC instead of assembly, and parsing stops at the first syntax error. Next we would add while loops, assembly output and web tabs for the tree, symbol table and TAC.

References

[1] Aho, Lam, Sethi, Ullman, Compilers: Principles, Techniques, and Tools, 2nd ed., Pearson, 2006.

[2] Python Software Foundation, Python 3 Documentation, https://docs.python.org/3/

[3] Pallets Projects, Flask Documentation, https://flask.palletsprojects.com/